Experiment 1: Implement Perceptron Algorithm using Scikit-learn

Aim / Objective

To implement and evaluate a simple Perceptron classifier on the Iris dataset using Scikit-learn.

Theory / Introduction

The Perceptron is one of the simplest types of artificial neural networks. It is a binary classifier that makes its predictions based on a linear predictor function combining a set of weights with the feature vector.

The Perceptron algorithm updates the weights based on the classification error for each sample. It is best suited for datasets that are linearly separable.

In this experiment, we apply the Perceptron to the well-known Iris dataset using scikit-learn's implementation and observe its behavior and performance.

Software / Tools Required

Python, Scikit-learn, Pandas, Matplotlib, Jupyter Notebook (optional)

Dataset Used

Iris Dataset (available from sklearn.datasets)

🔹 **About the Dataset**

The **Iris dataset** is a famous benchmark dataset in pattern recognition and machine learning. It consists of **150 flower samples**, divided equally among **three species** of Iris:

- *Iris-setosa*
- *Iris-versicolor*
- *Iris-virginica*

Each flower is described using **4 features**:

1. Sepal length (cm)
2. Sepal width (cm)
3. Petal length (cm)
4. Petal width (cm)

✅ **Available via Python:**

from sklearn.datasets import load_iris

📚 No need to download separately — it's a built-in dataset in scikit-learn.

Procedure / Steps

1. Import the necessary libraries.
2. Load the Iris dataset and prepare the feature and label arrays.
3. Preprocess the data (e.g., select only two classes and two features for binary classification).
4. Split the dataset into training and testing sets.
5. Train a Perceptron model on the training data.
6. Make predictions and evaluate the accuracy.
7. Visualize the decision boundary (optional).

Python Step-by-Step Code with Explanations

```
# Step 1: Import necessary libraries
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np

# Step 2: Load dataset and filter for binary classification (Setosa vs Versicolor)
iris = load_iris()
X = iris.data[:, [0, 2]]  # sepal length and petal length
y = iris.target

# Convert to binary classification problem (only class 0 and 1)
X = X[y != 2]
y = y[y != 2]

# Step 3: Train-test split and feature scaling
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1,
stratify=y)
sc = StandardScaler()
X_train_std = sc.fit_transform(X_train)
X_test_std = sc.transform(X_test)

# Step 4: Train Perceptron model
ppn = Perceptron(max_iter=40, eta0=0.1, random_state=1)
ppn.fit(X_train_std, y_train)

# Step 5: Predict and evaluate
```

```
y_pred = ppn.predict(X_test_std)
print(f'Accuracy: {accuracy_score(y_test, y_pred):.2f}')

# Step 6: Visualize decision boundary (optional)
x_min, x_max = X_train_std[:, 0].min() - 1, X_train_std[:, 0].max() + 1
y_min, y_max = X_train_std[:, 1].min() - 1, X_train_std[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))
Z = ppn.predict(np.array([xx.ravel(), yy.ravel()]).T)
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X_train_std[:, 0], X_train_std[:, 1], c=y_train, marker='o')
plt.title('Perceptron Decision Boundary')
plt.xlabel('Sepal length (standardized)')
plt.ylabel('Petal length (standardized)')
plt.show()


# Step 6b: Visualize decision boundary for test data
x_min, x_max = X_test_std[:, 0].min() - 1, X_test_std[:, 0].max() + 1
y_min, y_max = X_test_std[:, 1].min() - 1, X_test_std[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))
Z = ppn.predict(np.array([xx.ravel(), yy.ravel()]).T)
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X_test_std[:, 0], X_test_std[:, 1], c=y_test, marker='s', edgecolor='k')
plt.title('Perceptron Decision Boundary (Test Data)')
plt.xlabel('Sepal length (standardized)')
plt.ylabel('Petal length (standardized)')
plt.show()
```

Result / Output

The model prints the accuracy score and shows a decision boundary plot visualizing classification regions.

Conclusion

The Perceptron classifier was successfully implemented and achieved good accuracy on a linearly separable subset of the Iris dataset.

Viva Questions (2–3 per experiment)

1. What is the limitation of the Perceptron algorithm?
2. Why did we use only two classes and two features from the Iris dataset?
3. What is the effect of feature scaling on Perceptron training?